

The Character Sheet

A companion app and content admin for ASoHaV — plan for approval, draft 3

ALL 17 SCHEMA CONFLICTS RESOLVED • AWAITING SIGN-OFF TO BEGIN PHASE 0

Draft 2 raised seventeen conflicts between your two schemas, the five rules documents, and the paper sheet. **You've answered all seventeen.** This draft turns them into the model I'll actually build, so §3 is now a record of decisions rather than a list of questions.

Two of your answers changed the architecture rather than just a field, and they're worth reading before the rest: **Bonds and Rapport are shared objects, not character properties (§2)** — which is the correct model, and also the thing local storage handles worst. And **consolidating repeated fields into base classes** (Issue 6) let me collapse a fair amount of the schema.

What's left open is no longer conflicts — it's **six entities that don't exist yet** and that the sheet needs (§4). I've proposed shapes for all of them; I need you to correct or bless those, and that's the last thing standing between here and Phase 1.

1 Architecture

Local-first, structured as though the server already exists

Unchanged from draft 2, and your Issue 4 answer strengthens it in one direction and weakens it in another — see the catch below. The three disciplines:

1. **Every object carries a GUID** — your schema already does this — plus `ownerId`, `createdAt`, `updatedAt`. These now live on a `BaseEntity` class (per Issue 6) rather than being repeated.
2. **All reads and writes go through one adapter** (`store.js`), async from day one: `await store.get('characterSheet', id)`. `localStorage` is implementation one; `fetch()` is implementation two and touches no other file.
3. **The JSON export format is the API contract** — what the eventual database stores, and what your friend's GM tooling reads.

THE CATCH, NOW SHARPER

You confirmed **Bonds are shared between two PCs and Rapport is a pool shared by the whole party**. That's the right model. It also means the two systems your Advancement panel is built around *have no honest local implementation* — a shared pool stored in four browsers is four different numbers.

v1 will handle this by making the sharing **visible rather than pretended**: Rapport and each Bond render with a "shared — confirm at the table" marker, edits are logged with a timestamp, and import reconciles by showing both values and asking which is right rather than silently picking. It's honest, and it's usable for playtesting. It is not correct, and it won't be until there's a server.

Everything else on the sheet — Virtues, Conditions, Statuses, Themes, Load, Armor, Potential — is genuinely fine locally.

2 The reconciled data model

Three layers now, not two. The middle one is new and comes directly from your Issue 4 answer.

Base classes NEW — ISSUE 6

CLASS	FIELDS
BaseEntity	Id (GUID), CreatedAt, UpdatedAt, CreatedBy, UpdatedBy — everything inherits this.
LibraryEntity	: BaseEntity + Name, Description . Every authored object.
MoveBase	: LibraryEntity + Virtue, Results, PlayerVariantResults?. Move and AdventureMove both extend it and add nothing (Issue 9 removed Rank).

Per your instruction, I've also dropped every field reachable through a relationship:

CharacterSheet.Name and .PlayerName (reach via Character), VirtueValue.VirtueName (→ VirtueId), CharacterTheme.Name/Description (→ ThemeId).

Library — authored in the admin panel

OBJECT	STATE	SHAPE
Virtue	as-is	Tagline, Essence, UsageHelperText . Five records.
Condition	as-is	ClearAction, VirtueId, RollPenalty . Floor of -3 total confirmed (Issue 14).
Move / AdventureMove	changed	: MoveBase . Results {Tier1 = miss, Tier2 = 7-9, Tier3 = 10+} (Issue 7). Optional PlayerVariantResults of the same shape for <i>Sway the Spirit</i> , <i>Strike a Nerve</i> . Rank deleted.
Result	changed	Description, Options: List<Option>, ChooseCount: int (Issue 8). ChooseCount 0 means plain prose.
Theme → Quest	renamed	"Theme" everywhere, in code and UI (Issue 5). Quest gains StaleAfter? as a reserved field for the penalty you're planning.
Item	changed	+ LoadCost: int (0-2) (Issue 2), + GrantsArmorTypeId? , existing Charges .
Skill	as-is	Effect: string . Modifier field deferred (Issue 16).
Clock	changed	+ Polarity: ENUM(Positive, Negative, Neutral) (Issue 3). Owner → reference, not string (Issue 17). Max 5 → 6 (Issue 1). Private: bool retained.
Ability	to define	<u>§4</u>
ArmorType	to define	<u>§4</u>
Advancement	to define	<u>§4</u> — four tiers (Issue 10), global list, repeatable where flagged

Shared state — owned by the table, not by one sheet NEW LAYER — ISSUE 4

OBJECT	SHAPE AND WHY IT LIVES HERE
Party	CampaignId, MemberCharacterIds[], Rapport: int (0-5), RapportAdvancementsTaken[], PartyLevel: int . Rapport is a single pool that any member spends, so it is stored exactly once. It moves off CharacterAdvancement entirely.
Bond	CharacterAId, CharacterBId, KinTrack: int (0-5), BondLevel: int (0-5), BondMoves: [{Level, Text, AuthoredAt}] . One record per pair, not one per person. Forged: bool → BondLevel: int . Spending Kin and forging are symmetric by construction because there's only one record to change. Bond Moves are written by the two players , so they are free text stored here — not a library object.

Play state — one per character

OBJECT	STATE	SHAPE
Character	changed	The lightweight public identity other sheets reference: <code>Name</code> , <code>PlayerName</code> , <code>UserId</code> , <code>CampaignId</code> .
CharacterSheet	changed	<code>CharacterId</code> , <code>Level</code> : int (Issue 10), <code>Looks</code> , + the sub-objects below. Duplicated name fields removed. No <code>PlaybookId</code> — playbooks are out.
VirtueValue	changed	<code>VirtueId</code> , <code>Score</code> , <code>ConditionMarked</code> : bool.
CharacterStatus	changed	<code>Name</code> , <code>Rank</code> : int (1–6), <code>Polarity</code> , <code>LinkedToIds[]</code> , <code>AffectedByIds[]</code> . Rank is a live magnitude (Issue 1). Targeted statuses keep free-text names for now — <i>noted as a future update: select a real Character</i> (Issue 3).
CharacterTheme	changed	<code>ThemeId</code> , <code>AcceptedQuests</code> : [{ <code>QuestId</code> , <code>Completed</code> , <code>AcceptedAt</code> }] . No copied strings — admin edits now reach every character (Issue 5).
CharacterLoad	changed	<code>Tier</code> : ENUM(Light, Normal, Heavy), <code>EffectiveCapacity</code> = <code>BaseCapacity</code> + <code>Might</code> (Issue 2), <code>LatchedUntilCamp</code> : bool.
CharacterItem	new	<code>ItemId</code> , <code>Carried</code> : bool, <code>ChargesUsed</code> : int (Issue 2). Replaces <code>List<Item></code> — a sheet references definitions, it doesn't own them.
CharacterArmor	new	<code>ArmorTypeId</code> , <code>Used</code> : bool, <code>SourceId</code> . On the paper sheet, absent from both diagrams.
CharacterAdvancement	changed	Now <i>only</i> <code>Potential</code> : int (0–5), <code>PotentialAdvancementsTaken[]</code> . Kin moved to <code>Bond</code> , Rapport moved to <code>Party</code> .
Campaign / User / Membership	new	<code>Membership</code> { <code>UserId</code> , <code>CampaignId</code> , <code>Role</code> : ENUM(GM, Player), <code>CharacterId?</code> }. GM role suppresses the sheet.

3 Resolutions

Your answers, recorded. Nothing here needs further input.

#	ISSUE	RESOLUTION
1	Status Rank	Magnitude, 1–6. Schema max raised 5 → 6. Built as a bidirectional pip stepper, not a fill-toward-threshold clock.
2	Load	<code>BaseCapacity + Might</code> confirmed. <code>Item.LoadCost</code> added. <code>Carried</code> flag added.
3	Status polarity	Polarity enum added. Targeted statuses stay free-text; flagged in the schema as a future character-reference.
4	Bonds & Rapport	Both promoted to shared objects. Bond is per-pair; Rapport lives on Party. See §2 .
5	Theme naming	" Theme " everywhere. Reference-based <code>CharacterTheme</code> accepted. Stale-request penalty reserved for later.
6	Duplication	Base classes introduced; every relationally-reachable field dropped.
7	Move tiers	Tier1 = miss, Tier2 = 7–9, Tier3 = 10+. <code>PlayerVariantResults</code> added.
8	Result options	Added now. <code>Options[]</code> + <code>ChooseCount</code> .
9	<code>AdventureMove.Rank</code>	Typo — removed.
10	Advancement rule	Four-tier version. <code>Level</code> added to CharacterSheet, <code>PartyLevel</code> to Party.
11	Kin / Kith	Kin.
12	Potential / XP	Potential.
13	Ability / Virtue	Virtue where the rules meant a stat. <i>Ability</i> is now free to be the Playbook-power entity.
14	Condition floor	–3 total confirmed. Sheet shows the effective modifier per Virtue.
15	Dishonored	Badge/warning at 5/5 Conditions.
16	Skill modifier	Deferred. Carried as a TODO in the schema doc.
17	<code>Clock.Owner</code>	Now a reference.
—	Link to / Affected by	Placeholder buttons, wired to nothing, visually marked as not-yet-functional.
—	Moves drawer	In v1.

#	ISSUE	RESOLUTION
—	Build order	Sheet first, admin panel after.

4 The four new entities

Playbooks and Party Playbooks are **out** per your feedback, which removes two entities and simplifies three others: Advancements become a single global list gated by tier rather than a per-playbook one, Abilities are no longer owned by a playbook, and Bond Moves stop being library content altogether.

That leaves four things to define. Three are small. The fourth — Ability — is the one that needs real thought, because you've asked it to carry the full range of PbtA effects.

ArmorType SETTLED

: LibraryEntity + Key: ENUM(Physical, Heavy, Special) . Special is one type, not a family. Camp refresh is all-or-nothing, so it's a single action on the Armor panel rather than a per-box reset.

Advancement SETTLED

: LibraryEntity + Tier: int (1-4), Track: ENUM(Potential, Rapport), Repeatable: bool, MaxTimes: int?, Effect .

One global list, filtered by track and by your unlocked tier. Repeatable handles "some can be taken more than once"; MaxTimes is nullable so a repeatable advancement can be unlimited or capped. In the picker, a non-repeatable advancement you already hold greys out rather than disappearing — you can still read what you took.

BondMove NO LONGER A LIBRARY OBJECT

Since the two PCs write it together, there is nothing to author in the admin panel. It becomes play state on the shared Bond record: BondMoves: [{Level: int, Text: string, AuthoredAt}]. When a pair reaches a new Bond Level, the sheet opens a text field on both their sheets and stores one copy.

Ability NEEDS THE MOST STRUCTURE

You asked for something flexible enough for PbtA — permanent Virtue boosts, +2 Ongoing, and "other things". The trap here is obvious: model it too loosely and it's a wiki page with no mechanical hooks; model it too tightly and the first ability you write won't fit.

THE APPROACH

Prose is authoritative; structure is optional metadata. Every Ability carries human-readable rules text that is always displayed and always correct. Alongside it sits a list of *structured effects* — zero or more — that a future rules engine can act on, and that the sheet uses today only for things it can safely do without judgement.

This means an ability that doesn't fit the structure still works perfectly: you write the prose, add no effects, and it displays like any other. Nothing is ever blocked on the model being expressive enough. And when you later build the engine, the abilities that *do* carry effects light up without a migration.

```
Ability : LibraryEntity + RulesText: string (required, authoritative), Acquisition:  
ENUM(Starting, Advancement, Item, Bond, Other), Tags: string[], Effects: AbilityEffect[] .
```

AbilityEffect

A discriminated shape — `kind` selects which of the nullable fields apply.

KIND	FIELDS USED	COVERS
VirtueBoost	VirtueId, Value, Duration	Permanent stat raises; also temporary ones via Duration.
RollBonus	Value, Duration, AppliesToMoveId?, AppliesToVirtueId?, TriggerText	+1 Forward, +2 Ongoing, "+1 when you..."
GrantArmor	ArmorTypeId, Count	Armor boxes from abilities.
GrantMove	MoveId	Abilities that hand you a new move.
ModifyMove	MoveId, ReplacementResults?, AddedOptions[]	"When you Take a Risk, add this to the 7–9 list" — very common in PbtA.
GrantStatus	StatusName, Polarity, Rank, TriggerText	Abilities that apply Statuses.
ResourceChange	Resource: ENUM(Load, Potential, Kin, Rapport, Recovery), Value	Capacity changes, extra advancement ticks.
Hold	Count, SpendText	"Hold 3, spend 1 to..."
Narrative	Text only	The escape hatch. Fiction-first abilities with no numbers.

Shared by all kinds: `Duration: ENUM(Permanent, Ongoing, Forward, Instant, WhileConditionHolds)`, `TriggerText: string?`, and `Uses/RechargeOn` for limited abilities.

What the sheet does with this in v1: applies `VirtueBoost` at `Permanent`, materialises `GrantArmor` boxes with the ability as their source, and adds `GrantMove` entries to the Moves drawer. Everything else *displays* as an annotation next to the relevant panel — a "+2 Ongoing to Mettle" chip beside the Virtue — without altering any number. Consistent with the enforcement line: the sheet shows you what applies and lets you do the arithmetic.

Since abilities are no longer playbook-owned, one open thread: **how does a character acquire them at creation?** The `Acquisition` enum records the answer per ability, but nothing yet says how many a new character picks. Not blocking — v1 lets you add any ability to your sheet freely — but it'll need a rule eventually.

5 The character sheet

All data preserved, layout reorganised for screen. Responsive across phone, tablet, desktop.

Layout

Phone — one scrolling column under a sticky tab bar: *Sheet · Status · Theme · Kit · Growth*, with the Moves drawer reachable from every tab. The hot path at the table — mark a Condition, move a Status rank, tick Armor — is one tap from anywhere.

Tablet and desktop — two columns, no tabs. A persistent left rail holds identity: name, level, Looks, and the Virtues/Conditions table, which never scrolls away because every roll touches it. The right column scrolls: Statuses & Armor → Theme → Load → Advancement.

Panels

PANEL	TREATMENT
Virtues & Conditions	The table is the control. Tap a Condition to mark it: the row takes an ink wash and reveals its Clear Action inline. The effective modifier shows beside the score — a Conditioned Wit +1 reads +1 → -1 — floored at -3. At 5/5 marked, a Dishonored badge appears on the panel header. Essence and usage text collapse behind an ⓘ disclosure.
Statuses	Two groups, Positive and Negative, driven by polarity. Each row is a name plus a six-pip magnitude stepper, tappable up and down, clearing at zero. Negative pips ink red-brown, Positive gold. <i>Link to</i> and <i>Affected by</i> render as visibly-inert placeholder buttons. A Camp action offers the differential clear (2D6 Negative / 1D6 Positive) as a prompt, not an automation.
Armor	A row per box: Type · Used · Source, source auto-filled from the granting Ability or Item. <i>Used</i> is a single tap — it happens mid-hit. "Refresh all" at Camp.
Theme	Pick a Theme; its description and Starting Quest render live from the library. Accepted Quests list beneath with completion checks and an accepted-date, ready for the staleness rule when you design it.
Load	Light / Normal / Heavy as a segmented choice, each showing its computed capacity (base + Might). Below, the full inventory: a carry toggle per item, its Load cost, and a charge pip row where relevant. Carried weight totals against the chosen tier; over-limit is a soft warning. Latching until Camp is shown as state, not enforced as a lock.
Advancement	Potential as five pips on the sheet. Filling it opens a tier-filtered picker over the global Advancement list and resets to zero, as you specified; repeatable advancements stay in the list, others grey out once taken. Kin and Rapport render here too but are drawn from the shared objects, each carrying a "shared — confirm at the table" marker. The Bond table lists one row per party member: Kin track, Bond Level, and the Bond Moves — the latter authored by the two players in a text field when a Bond Level is reached.
Moves drawer	Searchable, filtered by Virtue, showing all three tiers and the Player Variant where one exists. Choice-lists render as lists now that Result carries options. Read-only — no rolling in v1.

The enforcement line

Shape enforced, legality warned. A track has five pips; a rank can't go below zero; filling a track prompts a choice. But you can exceed your Load, break the Virtue spread, or take an unearned advancement — each shows a quiet warning and lets you proceed. Confirmed in draft 2, restated here because it governs every panel above.

6 The admin panel

- **CRUD on every library type**, generated from schema definitions rather than hand-built per type — so the six new entities in §4 cost a config entry each, not a screen each.
- **Bulk JSON import/export**, whole-library and per-type. Also the seeding path for your existing docs.
- **Change history** — `{who, when, objectType, objectId, before, after}` per write, with a per-object timeline, field-level diff, and a global feed. *"Who" is self-declared until real auth exists — v1 asks which of you is at the keyboard.*
- **Reference validation** — orphaned Quests, Advancements with no tier, Items granting a nonexistent ArmorType.
- **Preview on sheet** — render any object as a player will see it, without leaving the editor.

7 Visual direction

Classical, pushed toward high fantasy. Cormorant Garamond over Lora, hairline rules, gold as stroke rather than fill — a printed book rather than a web app. The twist, kept disciplined:

- Ground warmed toward **aged parchment**, near-black toward ink-brown.
- Gold carries **illuminated rule-work** at section heads, with versal capitals on panel titles.
- **Every track drawn as ink** — hollow circles that fill. No checkboxes, no toggles, no progress bars.
- A marked Condition is an **ink wash across the row**. Negative Statuses bleed red-brown; Positive take gold.
- No parchment textures, no wood, no dragons, no torn edges. The fantasy is typographic.

Extended from the Classical tokens rather than forked, so the design system stays the source of truth for both apps.

8 Phasing

PHASE	DELIVERABLE	GATED ON
0	Schema document written out in full, incorporating every resolution above. <code>store.js</code> contract. Seed library.	resolved
1	Character sheet rendering a seeded character. Proves the model end to end.	Phase 0
2	Full interactivity: conditions, status magnitudes, quests, load declaration, advancement pickers, Moves drawer.	Phase 1
3	Admin panel: CRUD, import/export, change log, validation, preview.	Phase 0
4	Campaign shell, roster, GM-role suppression, export/import handoff.	Phases 2, 3
5	Handoff package for Claude Code — resolved schema, tokens, working prototypes, spec.	Phase 4
Later	Dice roller, combat, real backend, GM tooling integration, character-referenced statuses, Skill modifiers.	Rules work on your side

ON THE CLAUDE CODE HANDOFF

Restating draft 2's recommendation because it still holds: **hand off after Phase 2, not now**. What a developer needs from us is a resolved model plus a prototype demonstrating the interactions. We now have the former — this document is most of it — but not the latter. Building the sheet first means the handoff ships with something that provably works rather than something that reads well.

9 Risks

1. **Shared state is the one genuine weakness.** Now that Bonds and Rapport are correctly modelled as shared, the gap between the model and what local storage can honour is explicit. v1 makes it visible rather than hiding it. It resolves only with a server.

2. **Combat will reshape the sheet.** Statuses, Armor and Load are all combat-adjacent. Panels are built independently so a Combat panel drops in — but expect churn, and don't over-polish before it lands.
3. **The Ability effect model is the one speculative piece.** §4 proposes a flexible structure covering the PbtA patterns in your rules, but you have not written the abilities yet — the first real batch will test whether it holds.
4. **The rules docs are still drafts.** Library content lives in data, not markup, so a rules change is an import rather than a rebuild.
5. **Two people authoring in two browsers will diverge.** Keep the exported library JSON somewhere shared — a drive folder or a git repo — and treat one export as canonical.

Approve §4 — or tell me to use placeholder library data — and I'll start Phase 0.